Documentation: DigplexQ

Heitor Baldo*

Contents

*hbaldo@usp.br

1 Introduction

DigplexQ is a Python package to perform computations with digraph-based complexes (directed flag complexes and path complexes). It is an "adjacency matrix-centered" package since it was designed so that the user can perform all computations just by entering an adjacency matrix as input.

This package implements several quantitative methods for the analysis of digraph-based complexes, mainly based on concepts from directed Q-Analysis [1].

2 Installation

The DigplexQ package can be installed through the PIP package manager via the `pip` command:

```
pip install digplexq
```

It is available in the PyPi repository at https://pypi.org/project/digplexq (current version 0.0.7).

3 Dependencies

The following Python packages are required:

- numpy
- scipy
- networkx
- gtda
- persim
- hodgelaplacians

4 Basic Usage

Here is a minimal example of using DigplexQ:

```
from digplexq.directed_q_analysis import *
from digplexq.digraph_based_complexes import *
from digplexq.structure_based_simplicial_measures import *
from digplexq.random_digraphs import *
from digplexq.utils import *

M = directed_erdos_renyi_GnM_model(20, 40, weight=False)
M = remove_double_edges(M) #remove double edges.

#Directed flag complex:
DFC = DirectedFlagComplex(M, "by_dimension_with_nodes")

#Maximal directed simplices:
maxsimp = MaximalSimplices(DFC)

#q-Adjacency matrix:
fast_q_adjacency_matrix(M, q=1)

#in-q-degree centrality
in_q_degree_centrality(M, q=1, results="nodes")
```

5 Modules Overview

5.1 digplexq.digraph_based_complexes

- `DirectedFlagComplex(A: NumPy matrix)`: Returns the directed flag complex of a given digraph (A must be the adjacency matrix of the digraph).
- `PathComplex(A: NumPy matrix)`: Returns the path complex of a given digraph (A must be the adjacency matrix of the digraph).

5.2 digplexq.directed_q_analysis

- `q_adjacency_matrix(DFC: array, q: int)`: Returns the q-adjacency matrix of a directed flag complex.

5.3 digplexq.structure_based_simplicial_measures

- `average_shortest_q_walk_length(A: NumPy matrix, q: int)`: Returns the average shortest q-walk length of a digraph.
- `in_q_degree_centrality(A: NumPy matrix, q: int)`: Returns the in-q-degree centrality of a digraph.
- `out_q_degree_centrality(A: NumPy matrix, q: int)`: Returns the out-q-degree centrality of a digraph.

6 Examples

6.1 Structure-Based Simplicial Measures

```
M = directed_erdos_renyi_GnM_model(20, 40, weight=False)
M = remove_double_edges(M) #remove double edges.

#in-q-degree centrality (for q=1)
in_q_degree_centrality(M, q=1, results="nodes")

#out-q-degree centrality (for q=1)
out_q_degree_centrality(M, q=1, results="nodes")

#average shortest q-walk length (for q=0)
average_shortest_q_walk_length(M, q=0)
```

References

[1] Baldo, H. (2024). Towards a Quantitative Theory of Digraph-Based Complexes and its Applications in Brain Network Analysis. [PhD Thesis, University of São Paulo] *arXiv:2409.09862*

7 License